

wasserstein_arrangement_code

April 12, 2026

This Jupyter notebook displays sample code used in the calculation of the example metrics in the strict and generic case on 5 points in Appendix B of [1]. The dataset on 6 Points can be found on Zenodo [2]. An excerpt of this code is part of the PhD thesis [3].

The following are the packages we need for the computations. Note that since there are methods implemented by multiple packages (like `OSCAR` and `Graphs`), if loaded simultaneously the package needs to be added as a prefix to the method (e.g. `Graphs.nv()`).

```
[1]: using Oscar
      using Graphs
      using CountingChambers
```

```
  /_/_\ /_/_\ /_/_\ /_/_\ |_/_/_\ | Combining and extending ANTIC, GAP,
|_|_| |\_/_\ |\_/_\ |\_/_\ /_/_\ |_/_/_\ | Polymake and Singular
\_/_/_\ \_/_/_\ \_/_/_\ \_/_/_\ |_/_/_\ | Type "?Oscar" for more information
o-----o-----o-----o-----o | Documentation: https://docs.oscar-
system.org
  S Y M B O L I C   T O O L S   | Version 1.7.2
```

Loading images 1.3.3 (Minimal and Canonical images)
by Christopher Jefferson (<http://caj.host.cs.st-andrews.ac.uk/>),
Markus Pfeiffer (<http://www.morphism.de/~markusp/>),
Rebecca Waldecker (<http://conway1.mathematik.uni-halle.de/~waldecker/>), and
Eliza Jonauskyste (ej31@st-andrews.ac.uk).

maintained by:

Christopher Jefferson (<http://caj.host.cs.st-andrews.ac.uk/>).

Homepage: <https://gap-packages.github.io/images/>

Report issues at <https://github.com/gap-packages/images/issues>

The basic methods to enumerate the chambers of the arrangement are listed below. Their purposes are as follows:

AllCycles

A method to enumerate all cycles of a given graph G .

WasserteinMatroid

A method constructing the matroid consisting of all cycles of even length of the complete graph for a given number of vertices.

KRWPolytope

A method to construct the KRW-polytope of a given metric.

MetricCone

A method to encode the metric cone (see [1], Section 2)

```
[2]: function AllCycles(G)
    DG = SimpleDiGraph(Graphs.nv(G))
    for e in Graphs.edges(G)
        Graphs.add_edge!(DG, e.dst, e.src)
        Graphs.add_edge!(DG, e.src, e.dst)
    end
    cycs = filter(k->length(k)>2,simplecycles(DG))
    return cycs
end
```

[2]: AllCycles (generic function with 1 method)

```
[3]: function WassersteinMatroid(dim, return_matroid)
    G = Graphs.complete_graph(dim)
    cyclelist = AllCycles(G)
    evencycls = filter!(x-> length(x)% 2 == 0, cyclelist)
    listecycles = []
    edgelist = collect(combinations(1:dim, 2))
    for cycle in evencycls
        permvect = vcat([1], reverse(2:length(cycle)))
        if !(cycle[permvect] in listecycles)
            push!(listecycles, cycle)
        end
    end
    mat = zeros{Int, length(edgelist), length(listecycles))
    counter = 1
    for cycle in listecycles
        paritybit = 1
        m = length(cycle)
        for i in (1:m)
            if i < m
                pair = sort(cycle[[i,i+1]])
            else
                pair = sort(cycle[[1,m]])
            end
            index = findfirst(x-> x==pair , edgelist)
            mat[index, counter] = paritybit
            paritybit = paritybit * (-1)
        end
        counter = counter + 1
    end
end
```

```

    if return_matroid
        return [matrix(Int, mat), matroid_from_matrix_columns(matrix(QQ, mat))]
    else
        return mat
    end
end

```

[3]: WassersteinMatroid (generic function with 1 method)

The `return_matroid` parameter lets us choose between the matroid itself, needed for subsequent methods, and the matrix underlying it to be able to actually see its structure.

```
[4]: mat5 = WassersteinMatroid(5, false)
```

```
[4]: 10×15 Matrix{Int64}:
 1  1  1  1  1  1  0  0  0  0  0  0  0  0  0
 0  0 -1  0 -1  0  1  1  1  1  0  0  0  0  0
-1  0  0  0  0 -1 -1  0  0 -1  1  1  0  0  0
 0 -1  0 -1  0  0  0 -1 -1  0 -1 -1  0  0  0
-1 -1  0  0  0  0 -1 -1  0  0  0  0  1  1  0
 0  0 -1 -1  0  0  1  0  0  0 -1  0  0 -1  1
 0  0  0  0 -1 -1  0  1  0  0  1  0 -1  0 -1
 1  0  1  0  0  0  0  0 -1  0  0 -1 -1  0 -1
 0  1  0  0  1  0  0  0  0 -1  0  1  0 -1  1
 0  0  0  1  0  1  0  0  1  1  0  0  1  1  0

```

We can use the `CountingChambers` package to get the number of chambers:

```
[5]: number_of_chambers(mat5)
```

[5]: 882

The following methods are used to construct the KRW polytope by first constructing the matrix of the vectors whose convex hull forms the polytope. Since we employ methods from the `OSCAR` package and of `polymake`, we need the object encoded in both these languages.

```
[6]: function KRWPolytopeMatrix(metric, n)
    mat = zeros(QQ, n*(n-1), n)
    counter = 1
    for i in combinations(1:n, 2)
        if ndims(metric) == 1
            value = metric[counter]
        elseif ndims(metric) == 2
            value = metric[i[1], i[2]]
        end
        mat[counter*2-1, i[1]] = 1//value
        mat[counter*2-1, i[2]] = -1//value
        mat[counter*2, i[1]] = -mat[counter*2-1, i[1]]
    end
end

```

```

        mat[counter*2,i[2]] = -mat[counter*2-1, i[2]]
        counter = counter + 1
    end
    return mat
end

function KRWPolytopeOscar(metric, n)
    mat = KRWPolytopeMatrix(metric, n)
    return convex_hull(mat)
end

function KRWPolytopePolymake(metric, n)
    mat = KRWPolytopeMatrix(metric, n)
    m = size(mat,1)
    one_vector = ones(Int,m,1)
    mat = hcat(one_vector,mat)
    return Polymake.polytope.Polytope(POINTS=mat)
end

```

[6]: KRWPolytopePolymake (generic function with 1 method)

As an example we encode the metric with $\rho_{ij} = 1$ for all $i \neq j$. The corresponding polytope is the type A_4 root polytope, which we can compute with the `WasserteinPolytope` method.

```

[7]: n = [0 1 1 1 ; 1 0 1 1 ; 1 1 0 1 ; 1 1 1 0];
      P = KRWPolytopeOscar(n, 4)

```

[7]: Polyhedron in ambient dimension 4

```

[8]: function MetricConeMatrix(n)
    @assert n > 1
    edges = collect(combinations(1:n, 2))
    m = length(edges)
    mat = zeros(Int,m,m)
    for i in 1:m
        mat[i,i] = 1
    end
    S3 = symmetric_group(3)
    for tuple in combinations(1:n, 3)
        for pi in S3
            triple = [tuple[pi(i)] for i in 1:3]
            vect = repeat([0], outer = m)
            vect[findfirst(x -> x == sort([triple[1], triple[2]]), edges)] = 1
            vect[findfirst(x -> x == sort([triple[2], triple[3]]), edges)] = 1
            vect[findfirst(x -> x == sort([triple[1], triple[3]]), edges)] = -1
            mat = hcat(mat, vect)
        end
    end
end

```

```

        mat = transpose(mat)
        return return mat
end

function MetricCone(n)
    @assert n > 1
    mat = MetricConeMatrix(n)
    cone = Polymake.polytope.Cone(INEQUALITIES= mat)
    return cone
end

```

[8]: MetricCone (generic function with 1 method)

The main method `getUniqueGenericMetrics` constructs a hyperplane arrangement with the Wasserstein matroid as underlying matroid, finds an inner point in each chamber to construct the polytope and finally returns a list of unique elements. This is the way of enumerating the different unlabeled types. This method is very resource-intensive and thus we were only able to use it for $n \leq 5$, the computations for $n = 6$ were done with Jörg Rambau's Software TOPCOM [4]. The `polish_metric` method clears the denominators from the metrics that arise as rational numbers. This improves readability of sample metrics without modifying their combinatorial type.

```

[9]: function polish_metric(metric)
        oscar_metric = [QQ(d) for d in metric]
        oscar_metric = lcm([denominator(i) for i in oscar_metric])*oscar_metric
    end

    function get_non_isomorphic_polytopes(n, metric_list)
        polytope_list = []
        for metric in metric_list
            push!(polytope_list, KRWPolytopePolymake(metric, n))
        end
        unique_indices = []
        for i in 1:length(metric_list)
            krw_polytope = polytope_list[i]
            unique = true
            for j in 1:length(unique_indices)
                unique_krw_polytope = polytope_list[unique_indices[j]]
                if Oscar.Polymake.polytope.isomorphic(krw_polytope,
↪unique_krw_polytope)
                    unique = false
                    break
                end
            end
            if unique
                push!(unique_indices, i)
            end
        end
    end
end

```

```

    return [(polyhedron(polytope_list[i[1]]), polish_metric(metric_list[i[1]]))
    ↪for i in unique_indices]
end

function getUniqueGenericMetrics(n)
    HA = Polymake.fan.HyperplaneArrangement(
        HYPERPLANES=transpose(WassersteinMatroid(n,false)),
        SUPPORT = MetricCone(n))
    decomp = HA.CHAMBER_DECOMPOSITION
    m = decomp.MAXIMAL_CONES
    raymatrix = decomp.RAYS
    metric_list = []
    println("number of chambers: ", decomp.N_MAXIMAL_CONES)
    for i in 1:size(m)[1]
        cone = Polymake.polytope.Cone(INPUT_RAYS=raymatrix[m[i,:], :])
        metric = cone.REL_INT_POINT
        push!(metric_list, metric)
    end
    return get_non_isomorphic_polytopes(n, metric_list)
end

```

[9]: getUniqueGenericMetrics (generic function with 1 method)

```

[10]: M4 = getUniqueGenericMetrics(4)
@time M5 = getUniqueGenericMetrics(5)

```

```

number of chambers: 6
number of chambers: 882
17.696791 seconds (154.29 M allocations: 2.063 GiB, 2.66% gc time, 0.10%
compilation time)

```

```

[10]: 12-element Vector{Tuple{Polyhedron{QQFieldElem}, Vector{QQFieldElem}}}:
 (Polytope in ambient dimension 5, [8, 18, 15, 11, 14, 15, 15, 7, 11, 14])
 (Polytope in ambient dimension 5, [8, 20, 15, 13, 16, 19, 13, 9, 11, 16])
 (Polytope in ambient dimension 5, [8, 18, 19, 11, 18, 15, 15, 7, 11, 14])
 (Polytope in ambient dimension 5, [10, 22, 17, 11, 16, 15, 17, 9, 15, 20])
 (Polytope in ambient dimension 5, [6, 12, 8, 6, 9, 8, 8, 6, 8, 12])
 (Polytope in ambient dimension 5, [6, 11, 7, 6, 8, 9, 6, 6, 7, 11])
 (Polytope in ambient dimension 5, [10, 22, 15, 13, 16, 21, 11, 11, 13, 20])
 (Polytope in ambient dimension 5, [12, 24, 25, 11, 24, 17, 19, 11, 17, 24])
 (Polytope in ambient dimension 5, [7, 16, 11, 10, 11, 16, 10, 7, 10, 10])
 (Polytope in ambient dimension 5, [14, 30, 21, 17, 20, 15, 27, 13, 17, 26])
 (Polytope in ambient dimension 5, [7, 13, 8, 7, 9, 7, 10, 7, 8, 13])
 (Polytope in ambient dimension 5, [11, 20, 20, 11, 20, 11, 20, 11, 11, 20])

```

We save the polytopes with their corresponding metric in a JSON file in the Zenodo data base [2].

```

[31]: save("Polytopes_generic_metrics_5.mrdi", M5)

```

```
[32]: M5_loaded = load("Polytopes_generic_metrics_5.mrdi")
```

```
[32]: 12-element Vector{Tuple{Polyhedron{QQFieldElem}, Vector{QQFieldElem}}}:
(Polytope in ambient dimension 5, [8, 18, 15, 11, 14, 15, 15, 7, 11, 14])
(Polytope in ambient dimension 5, [8, 20, 15, 13, 16, 19, 13, 9, 11, 16])
(Polytope in ambient dimension 5, [8, 18, 19, 11, 18, 15, 15, 7, 11, 14])
(Polytope in ambient dimension 5, [10, 22, 17, 11, 16, 15, 17, 9, 15, 20])
(Polytope in ambient dimension 5, [6, 12, 8, 6, 9, 8, 8, 6, 8, 12])
(Polytope in ambient dimension 5, [6, 11, 7, 6, 8, 9, 6, 6, 7, 11])
(Polytope in ambient dimension 5, [10, 22, 15, 13, 16, 21, 11, 11, 13, 20])
(Polytope in ambient dimension 5, [12, 24, 25, 11, 24, 17, 19, 11, 17, 24])
(Polytope in ambient dimension 5, [7, 16, 11, 10, 11, 16, 10, 7, 10, 10])
(Polytope in ambient dimension 5, [14, 30, 21, 17, 20, 15, 27, 13, 17, 26])
(Polytope in ambient dimension 5, [7, 13, 8, 7, 9, 7, 10, 7, 8, 13])
(Polytope in ambient dimension 5, [11, 20, 20, 11, 20, 11, 20, 11, 11, 20])
```

The symmetric group S_n acts on the Wasserstein arrangement by permutation of the labels of the n points of the space, which in turn induces a permutation of the labels of the distances of the metric. Thus, if we want to compute the orbit of a chamber of the arrangement and the stabilizer, we first need to encode this symmetry group. This is what the function `symmetry_group_in_wasserstein` is for.

```
[13]: function map_permutation_to_Starget(n, mat, sigma)
    pairs = collect(combinations(1:n,2))
    permuted_pairs = [sort([sigma(p[1]),sigma(p[2])]) for p in pairs]
    sigma_on_pairs = perm([findfirst(x -> x==p, permuted_pairs) for p in pairs])

    final_perm = Vector{Int}()
    all_columns = [mat[:,k] for k in 1:size(mat,2)]
    for i in 1:size(mat,2)
        permuted_column = permuted(mat[:,i], sigma_on_pairs)
        new_index = findfirst(x -> x==permuted_column, all_columns)
        if typeof(new_index) != Int
            new_index = -1*findfirst(x -> x== -1*permuted_column, all_columns)
        end
        push!(final_perm, new_index)
    end
    abs_perm = perm([abs(i) for i in final_perm])
    # we use signed permutation matrix to account for sign flips
    pmatrix = permutation_matrix{ZZ}(abs_perm)
    for i in 1:size(mat,2)
        if sign(final_perm[i]) == -1
            pmatrix[i,:] = -1*pmatrix[i,:]
        end
    end
    return pmatrix
end
```

```

function symmetry_group_in_wasserstein(n, mat)
  Sn = symmetric_group(n)
  Starget = general_linear_group(size(mat,2),ZZ)
  G = gens(Sn)
  Gtarget = [map_permutation_to_Starget(n, mat, sigma) for sigma in G]
  emb = hom(Sn, Starget, G, Gtarget)
  H = emb(Sn)
  return H[1], emb
end

```

[13]: symmetry_group_in_wasserstein (generic function with 1 method)

```

[14]: function matrix_act(vect, mat)
  return mat*matrix(vect)
end

function stabilizer_from_metric(n, mat, H, metric)
  sign_pattern = transpose(mat)*metric
  sign_pattern = matrix([ZZ(sign(i)) for i in sign_pattern])
  O = orbit(H, matrix_act, sign_pattern)
  println("order of the orbit ", length(O))
  return stabilizer(O)[1]
end

```

[14]: stabilizer_from_metric (generic function with 1 method)

As an example, we compute the generic metrics for $n = 5$.

```

[15]: n = 5
H, emb = symmetry_group_in_wasserstein(n, mat5)
counter = 0
for i in 1:length(M5)
  println("index: ", i)
  println("metric: ", M5[i][2])
  stab = stabilizer_from_metric(5, mat5, H, M5[i][2])
  println("order of the stabilizer: ", order(stab))
  stab_n = preimage(emb,stab)[1]
  global counter += 120/order(stab_n)
  println("type of the stabilizer: ", describe(stab_n))
  println("generators of the stabilizer: ", gens(stab_n))
  println()
end
counter

```

```

index: 1
metric: QQFieldElem[8, 18, 15, 11, 14, 15, 15, 7, 11, 14]
order of the orbit 60
order of the stabilizer: 2

```

type of the stabilizer: C2
 generators of the stabilizer: PermGroupElem[(1,3)(2,4)]

index: 2
 metric: QQFieldElem[8, 20, 15, 13, 16, 19, 13, 9, 11, 16]
 order of the orbit 120
 order of the stabilizer: 1
 type of the stabilizer: 1
 generators of the stabilizer: PermGroupElem[]

index: 3
 metric: QQFieldElem[8, 18, 19, 11, 18, 15, 15, 7, 11, 14]
 order of the orbit 60
 order of the stabilizer: 2
 type of the stabilizer: C2
 generators of the stabilizer: PermGroupElem[(1,3)(2,4)]

index: 4
 metric: QQFieldElem[10, 22, 17, 11, 16, 15, 17, 9, 15, 20]
 order of the orbit 120
 order of the stabilizer: 1
 type of the stabilizer: 1
 generators of the stabilizer: PermGroupElem[]

index: 5
 metric: QQFieldElem[6, 12, 8, 6, 9, 8, 8, 6, 8, 12]
 order of the orbit 120
 order of the stabilizer: 1
 type of the stabilizer: 1
 generators of the stabilizer: PermGroupElem[]

index: 6
 metric: QQFieldElem[6, 11, 7, 6, 8, 9, 6, 6, 7, 11]
 order of the orbit 60
 order of the stabilizer: 2
 type of the stabilizer: C2
 generators of the stabilizer: PermGroupElem[(1,5)(3,4)]

index: 7
 metric: QQFieldElem[10, 22, 15, 13, 16, 21, 11, 11, 13, 20]
 order of the orbit 120
 order of the stabilizer: 1
 type of the stabilizer: 1
 generators of the stabilizer: PermGroupElem[]

index: 8
 metric: QQFieldElem[12, 24, 25, 11, 24, 17, 19, 11, 17, 24]
 order of the orbit 60

```

order of the stabilizer: 2
type of the stabilizer: C2
generators of the stabilizer: PermGroupElem[(2,5)(3,4)]

index: 9
metric: QQFieldElem[7, 16, 11, 10, 11, 16, 10, 7, 10, 10]
order of the orbit 30
order of the stabilizer: 4
type of the stabilizer: C2 x C2
generators of the stabilizer: PermGroupElem[(1,3)(2,4), (1,2)(3,4)]

index: 10
metric: QQFieldElem[14, 30, 21, 17, 20, 15, 27, 13, 17, 26]
order of the orbit 60
order of the stabilizer: 2
type of the stabilizer: C2
generators of the stabilizer: PermGroupElem[(1,3)(2,4)]

index: 11
metric: QQFieldElem[7, 13, 8, 7, 9, 7, 10, 7, 8, 13]
order of the orbit 60
order of the stabilizer: 2
type of the stabilizer: C2
generators of the stabilizer: PermGroupElem[(1,4)(3,5)]

index: 12
metric: QQFieldElem[11, 20, 20, 11, 20, 11, 20, 11, 11, 20]
order of the orbit 12
order of the stabilizer: 10
type of the stabilizer: D10
generators of the stabilizer: PermGroupElem[(1,2)(4,5), (1,4)(3,5)]

```

[15]: 882

The analogue methods can be used for the strict, non-generic case. We do not consider the maximal cones, but all others.

```

[25]: function getUniqueNonGenericStrictMetrics(n)
    m = binomial(n,2)
    MC = MetricCone(n)
    matrix_of_metric_cone = MetricConeMatrix(n)
    HA = Polymake.fan.
    ↪HyperplaneArrangement(HYPERPLANES=transpose(Matrix{Int64}(WassersteinMatroid(n,false))),
    ↪
    SUPPORT = MC);
    decomp = HA.CHAMBER_DECOMPOSITION
    Cs = decomp.CONES

```

```

println("done with cone decomposition")
raymatrix = decomp.RAYS
metriclist = []
for i in 1:(decomp.FAN_DIM-1)
    for j in 1:size(Cs[i])[1]
        co = Polymake.polytope.Cone(INPUT_RAYS = raymatrix[Cs[i][j,:],:])
        metric = co.REL_INT_POINT
        # check if the metric is strict:
        vec = matrix_of_metric_cone*metric
        if all(x-> x > 0, vec)
            push!(metriclist, metric)
        end
    end
end
println("done with metrics. found ", length(metriclist), " strict metrics.
↪")
return get_non_isomorphic_polytopes(n, metriclist)
end

```

[25]: `getUniqueNonGenericStrictMetrics` (generic function with 1 method)

```

[26]: S4 = getUniqueNonGenericStrictMetrics(4)
@time S5 = getUniqueNonGenericStrictMetrics(5)

```

1409.469376 seconds (4.20 G allocations: 58.290 GiB, 1.81% gc time, 0.00% compilation time)

[26]: 65-element Vector{Tuple{Polyhedron{QQFieldElem}, Vector{QQFieldElem}}}:
(Polytope in ambient dimension 5, [2, 2, 2, 2, 2, 2, 2, 2, 2])
(Polytope in ambient dimension 5, [2, 3, 3, 2, 3, 3, 2, 2, 3])
(Polytope in ambient dimension 5, [4, 6, 5, 5, 6, 5, 5, 5, 6])
(Polytope in ambient dimension 5, [3, 4, 3, 3, 3, 4, 4, 3, 4])
(Polytope in ambient dimension 5, [3, 5, 4, 4, 4, 5, 3, 3, 4])
(Polytope in ambient dimension 5, [2, 4, 4, 3, 4, 4, 3, 2, 3])
(Polytope in ambient dimension 5, [4, 8, 7, 5, 8, 7, 5, 5, 7])
(Polytope in ambient dimension 5, [4, 8, 7, 7, 8, 7, 7, 5, 6])
(Polytope in ambient dimension 5, [3, 5, 4, 3, 4, 5, 4, 3, 4])
(Polytope in ambient dimension 5, [6, 10, 7, 7, 8, 9, 9, 7, 10])
(Polytope in ambient dimension 5, [6, 10, 9, 7, 8, 7, 9, 5, 7])
(Polytope in ambient dimension 5, [3, 6, 5, 4, 5, 6, 3, 3, 4])
(Polytope in ambient dimension 5, [4, 7, 5, 5, 5, 7, 5, 4, 4])
(Polytope in ambient dimension 5, [6, 12, 9, 9, 10, 11, 7, 7, 10])
(Polytope in ambient dimension 5, [6, 12, 9, 7, 10, 7, 9, 7, 12])
(Polytope in ambient dimension 5, [3, 4, 3, 3, 4, 3, 3, 3, 4])
(Polytope in ambient dimension 5, [10, 14, 9, 9, 10, 11, 11, 9, 14])
(Polytope in ambient dimension 5, [6, 10, 7, 7, 8, 9, 5, 7, 10])
(Polytope in ambient dimension 5, [4, 10, 9, 7, 10, 9, 7, 5, 8])
(Polytope in ambient dimension 5, [3, 6, 5, 4, 5, 6, 5, 3, 4])

```

(Polytope in ambient dimension 5, [6, 12, 9, 7, 10, 11, 9, 7, 9, 12])
(Polytope in ambient dimension 5, [6, 12, 9, 9, 10, 11, 11, 7, 7, 10])
(Polytope in ambient dimension 5, [6, 12, 11, 7, 10, 9, 9, 5, 9, 10])

(Polytope in ambient dimension 5, [8, 16, 11, 9, 12, 7, 13, 9, 11, 16])
(Polytope in ambient dimension 5, [6, 14, 11, 9, 12, 13, 11, 7, 9, 12])
(Polytope in ambient dimension 5, [6, 14, 13, 9, 12, 11, 11, 5, 9, 10])
(Polytope in ambient dimension 5, [8, 16, 13, 9, 12, 13, 13, 7, 11, 14])
(Polytope in ambient dimension 5, [4, 9, 7, 6, 7, 9, 6, 4, 5, 7])
(Polytope in ambient dimension 5, [6, 16, 13, 11, 14, 15, 9, 7, 9, 12])
(Polytope in ambient dimension 5, [8, 18, 13, 11, 14, 17, 11, 9, 11, 16])
(Polytope in ambient dimension 5, [8, 18, 13, 13, 14, 17, 13, 9, 9, 14])
(Polytope in ambient dimension 5, [8, 16, 17, 9, 16, 13, 13, 7, 11, 14])
(Polytope in ambient dimension 5, [8, 18, 15, 9, 14, 11, 13, 7, 13, 16])
(Polytope in ambient dimension 5, [10, 20, 15, 11, 14, 13, 17, 9, 13, 18])
(Polytope in ambient dimension 5, [8, 18, 13, 9, 14, 13, 13, 9, 13, 18])
(Polytope in ambient dimension 5, [5, 9, 6, 5, 7, 7, 6, 5, 6, 9])
(Polytope in ambient dimension 5, [4, 8, 6, 4, 7, 5, 5, 4, 6, 8])
(Polytope in ambient dimension 5, [6, 11, 7, 6, 8, 7, 8, 6, 7, 11])
(Polytope in ambient dimension 5, [12, 22, 15, 11, 16, 15, 15, 11, 15, 22])
(Polytope in ambient dimension 5, [8, 16, 11, 9, 12, 15, 9, 9, 11, 16])
(Polytope in ambient dimension 5, [6, 10, 6, 6, 7, 8, 6, 6, 6, 10])
(Polytope in ambient dimension 5, [8, 18, 13, 11, 14, 17, 7, 9, 11, 16])
(Polytope in ambient dimension 5, [10, 20, 13, 13, 14, 19, 11, 11, 11, 18])
(Polytope in ambient dimension 5, [12, 22, 23, 11, 22, 15, 19, 11, 15, 22])
(Polytope in ambient dimension 5, [12, 26, 19, 15, 18, 11, 23, 11, 15, 22])
(Polytope in ambient dimension 5, [10, 20, 13, 11, 14, 11, 17, 11, 13, 20])

```

```

[27]: n = 5
H, emb = symmetry_group_in_wasserstein(n, mat5)
counter = 0

sorted_collection = Dict()
# [f_vector, [(metric, group)]]
for i in 1:length(S5)
    println("index: ", i)
    println("metric: ", S5[i][2])
    fvec = f_vector(S5[i][1])
    println("f vector: ", fvec)

    stab = stabilizer_from_metric(5, mat5, H, S5[i][2])
    println("order of the stabilizer: ", order(stab))
    stab_n = preimage(emb, stab)[1]
    global counter += 120/order(stab_n)
    println("type of the stabilizer: ", describe(stab_n))
    println("generators of the stabilizer: ", gens(stab_n))
    println()

```

```

    if haskey(sorted_collection, fvec)
        push!(sorted_collection[fvec], (S5[i][2], describe(stab_n)))
    else
        sorted_collection[fvec] = [(S5[i][2], describe(stab_n))]
    end
end
counter

```

```

index: 1
metric: QQFieldElem[2, 2, 2, 2, 2, 2, 2, 2, 2]
f vector: ZZRingElem[20, 60, 70, 30]
order of the orbit 1
order of the stabilizer: 120
type of the stabilizer: S5
generators of the stabilizer: PermGroupElem[(1,2,3,4,5), (1,2)]

```

```

index: 2
metric: QQFieldElem[2, 3, 3, 2, 3, 3, 2, 2, 3, 3]
f vector: ZZRingElem[20, 72, 94, 42]
order of the orbit 10
order of the stabilizer: 12
type of the stabilizer: D12
generators of the stabilizer: PermGroupElem[(1,2), (1,5), (3,4)]

```

```

index: 3
metric: QQFieldElem[4, 6, 5, 5, 6, 5, 5, 5, 5, 6]
f vector: ZZRingElem[20, 76, 102, 46]
order of the orbit 30
order of the stabilizer: 4
type of the stabilizer: C2 x C2
generators of the stabilizer: PermGroupElem[(1,2), (4,5)]

```

```

index: 4
metric: QQFieldElem[3, 4, 3, 3, 3, 4, 4, 3, 3, 4]
f vector: ZZRingElem[20, 72, 94, 42]
order of the orbit 10
order of the stabilizer: 12
type of the stabilizer: D12
generators of the stabilizer: PermGroupElem[(1,3)(2,4,5), (4,5)]

```

```

index: 5
metric: QQFieldElem[3, 5, 4, 4, 4, 5, 3, 3, 3, 4]
f vector: ZZRingElem[20, 82, 116, 54]
order of the orbit 60
order of the stabilizer: 2
type of the stabilizer: C2
generators of the stabilizer: PermGroupElem[(1,4)(2,3)]

```

index: 6
 metric: QQFieldElem[2, 4, 4, 3, 4, 4, 3, 2, 3, 3]
 f vector: ZZRingElem[20, 80, 114, 54]
 order of the orbit 15
 order of the stabilizer: 8
 type of the stabilizer: D8
 generators of the stabilizer: PermGroupElem[(1,2), (3,4), (1,3)(2,4)]

index: 7
 metric: QQFieldElem[4, 8, 7, 5, 8, 7, 5, 5, 7, 8]
 f vector: ZZRingElem[20, 80, 112, 52]
 order of the orbit 60
 order of the stabilizer: 2
 type of the stabilizer: C2
 generators of the stabilizer: PermGroupElem[(1,2)]

index: 8
 metric: QQFieldElem[4, 8, 7, 7, 8, 7, 7, 5, 5, 6]
 f vector: ZZRingElem[20, 80, 114, 54]
 order of the orbit 30
 order of the stabilizer: 4
 type of the stabilizer: C2 x C2
 generators of the stabilizer: PermGroupElem[(1,2), (4,5)]

index: 9
 metric: QQFieldElem[3, 5, 4, 3, 4, 5, 4, 3, 4, 5]
 f vector: ZZRingElem[20, 80, 112, 52]
 order of the orbit 60
 order of the stabilizer: 2
 type of the stabilizer: C2
 generators of the stabilizer: PermGroupElem[(2,5)]

index: 10
 metric: QQFieldElem[6, 10, 7, 7, 8, 9, 9, 7, 7, 10]
 f vector: ZZRingElem[20, 80, 112, 52]
 order of the orbit 60
 order of the stabilizer: 2
 type of the stabilizer: C2
 generators of the stabilizer: PermGroupElem[(4,5)]

index: 11
 metric: QQFieldElem[6, 10, 9, 7, 8, 7, 9, 5, 7, 8]
 f vector: ZZRingElem[20, 84, 122, 58]
 order of the orbit 60
 order of the stabilizer: 2
 type of the stabilizer: C2
 generators of the stabilizer: PermGroupElem[(1,3)(2,4)]

```

index: 12
metric: QQFieldElem[3, 6, 5, 4, 5, 6, 3, 3, 4, 5]
f vector: ZZRingElem[20, 84, 122, 58]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 13
metric: QQFieldElem[4, 7, 5, 5, 5, 7, 5, 4, 4, 6]
f vector: ZZRingElem[20, 84, 122, 58]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 14
metric: QQFieldElem[6, 12, 9, 9, 10, 11, 7, 7, 7, 10]
f vector: ZZRingElem[20, 84, 122, 58]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 15
metric: QQFieldElem[6, 12, 9, 7, 10, 7, 9, 7, 9, 12]
f vector: ZZRingElem[20, 84, 122, 58]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 16
metric: QQFieldElem[3, 4, 3, 3, 4, 3, 3, 3, 3, 4]
f vector: ZZRingElem[20, 80, 114, 54]
order of the orbit 30
order of the stabilizer: 4
type of the stabilizer: C2 x C2
generators of the stabilizer: PermGroupElem[(1,2), (4,5)]

index: 17
metric: QQFieldElem[10, 14, 9, 9, 10, 11, 11, 9, 9, 14]
f vector: ZZRingElem[20, 80, 114, 54]
order of the orbit 15
order of the stabilizer: 8
type of the stabilizer: D8
generators of the stabilizer: PermGroupElem[(4,5), (1,4)(3,5)]

```

index: 18
 metric: QQFieldElem[6, 10, 7, 7, 8, 9, 5, 7, 7, 10]
 f vector: ZZRingElem[20, 84, 122, 58]
 order of the orbit 60
 order of the stabilizer: 2
 type of the stabilizer: C2
 generators of the stabilizer: PermGroupElem[(1,5)(3,4)]

index: 19
 metric: QQFieldElem[4, 10, 9, 7, 10, 9, 7, 5, 7, 8]
 f vector: ZZRingElem[20, 84, 124, 60]
 order of the orbit 60
 order of the stabilizer: 2
 type of the stabilizer: C2
 generators of the stabilizer: PermGroupElem[(1,2)]

index: 20
 metric: QQFieldElem[3, 6, 5, 4, 5, 6, 5, 3, 4, 5]
 f vector: ZZRingElem[20, 86, 128, 62]
 order of the orbit 60
 order of the stabilizer: 2
 type of the stabilizer: C2
 generators of the stabilizer: PermGroupElem[(1,3)(2,4)]

index: 21
 metric: QQFieldElem[6, 12, 9, 7, 10, 11, 9, 7, 9, 12]
 f vector: ZZRingElem[20, 84, 122, 58]
 order of the orbit 120
 order of the stabilizer: 1
 type of the stabilizer: 1
 generators of the stabilizer: PermGroupElem[]

index: 22
 metric: QQFieldElem[6, 12, 9, 9, 10, 11, 11, 7, 7, 10]
 f vector: ZZRingElem[20, 84, 124, 60]
 order of the orbit 60
 order of the stabilizer: 2
 type of the stabilizer: C2
 generators of the stabilizer: PermGroupElem[(4,5)]

index: 23
 metric: QQFieldElem[6, 12, 11, 7, 10, 9, 9, 5, 9, 10]
 f vector: ZZRingElem[20, 86, 128, 62]
 order of the orbit 120
 order of the stabilizer: 1
 type of the stabilizer: 1
 generators of the stabilizer: PermGroupElem[]

index: 24
metric: QQFieldElem[8, 14, 11, 9, 10, 11, 13, 7, 9, 12]
f vector: ZZRingElem[20, 86, 128, 62]
order of the orbit 60
order of the stabilizer: 2
type of the stabilizer: C2
generators of the stabilizer: PermGroupElem[(1,3)(2,4)]

index: 25
metric: QQFieldElem[3, 7, 6, 5, 6, 7, 4, 3, 4, 5]
f vector: ZZRingElem[20, 86, 128, 62]
order of the orbit 60
order of the stabilizer: 2
type of the stabilizer: C2
generators of the stabilizer: PermGroupElem[(1,4)(2,3)]

index: 26
metric: QQFieldElem[4, 8, 6, 5, 6, 8, 5, 4, 5, 7]
f vector: ZZRingElem[20, 86, 128, 62]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 27
metric: QQFieldElem[2, 4, 3, 3, 3, 4, 3, 2, 2, 3]
f vector: ZZRingElem[20, 86, 128, 62]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 28
metric: QQFieldElem[6, 14, 11, 9, 12, 13, 7, 7, 9, 12]
f vector: ZZRingElem[20, 86, 128, 62]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 29
metric: QQFieldElem[6, 14, 11, 11, 12, 13, 9, 7, 7, 10]
f vector: ZZRingElem[20, 86, 128, 62]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 30
metric: QQFieldElem[8, 16, 11, 11, 12, 15, 11, 9, 9, 14]
f vector: ZZRingElem[20, 86, 128, 62]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 31
metric: QQFieldElem[8, 14, 15, 9, 14, 11, 13, 7, 9, 12]
f vector: ZZRingElem[20, 86, 128, 62]
order of the orbit 60
order of the stabilizer: 2
type of the stabilizer: C2
generators of the stabilizer: PermGroupElem[(1,3)(2,4)]

index: 32
metric: QQFieldElem[8, 16, 13, 9, 12, 9, 13, 7, 11, 14]
f vector: ZZRingElem[20, 86, 128, 62]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 33
metric: QQFieldElem[6, 14, 11, 7, 12, 9, 9, 7, 11, 14]
f vector: ZZRingElem[20, 86, 128, 62]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 34
metric: QQFieldElem[8, 16, 11, 9, 12, 11, 13, 9, 11, 16]
f vector: ZZRingElem[20, 86, 128, 62]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 35
metric: QQFieldElem[3, 5, 4, 3, 5, 4, 3, 3, 4, 5]
f vector: ZZRingElem[20, 84, 124, 60]
order of the orbit 60
order of the stabilizer: 2
type of the stabilizer: C2
generators of the stabilizer: PermGroupElem[(1,2)]

```

index: 36
metric: QQFieldElem[5, 8, 5, 5, 6, 6, 6, 5, 5, 8]
f vector: ZZRingElem[20, 84, 124, 60]
order of the orbit 60
order of the stabilizer: 2
type of the stabilizer: C2
generators of the stabilizer: PermGroupElem[(4,5)]

index: 37
metric: QQFieldElem[10, 16, 11, 9, 12, 13, 11, 9, 11, 16]
f vector: ZZRingElem[20, 86, 128, 62]
order of the orbit 60
order of the stabilizer: 2
type of the stabilizer: C2
generators of the stabilizer: PermGroupElem[(1,5)(3,4)]

index: 38
metric: QQFieldElem[4, 7, 5, 4, 6, 4, 5, 4, 5, 7]
f vector: ZZRingElem[20, 86, 128, 62]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 39
metric: QQFieldElem[12, 20, 13, 11, 14, 13, 15, 11, 13, 20]
f vector: ZZRingElem[20, 86, 128, 62]
order of the orbit 60
order of the stabilizer: 2
type of the stabilizer: C2
generators of the stabilizer: PermGroupElem[(1,4)(3,5)]

index: 40
metric: QQFieldElem[6, 12, 9, 7, 10, 11, 5, 7, 9, 12]
f vector: ZZRingElem[20, 86, 128, 62]
order of the orbit 60
order of the stabilizer: 2
type of the stabilizer: C2
generators of the stabilizer: PermGroupElem[(1,5)(3,4)]

index: 41
metric: QQFieldElem[8, 14, 9, 9, 10, 13, 9, 9, 9, 14]
f vector: ZZRingElem[20, 86, 128, 62]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

```

index: 42
 metric: QQFieldElem[8, 16, 11, 11, 12, 15, 7, 9, 9, 14]
 f vector: ZZRingElem[20, 86, 128, 62]
 order of the orbit 120
 order of the stabilizer: 1
 type of the stabilizer: 1
 generators of the stabilizer: PermGroupElem[]

index: 43
 metric: QQFieldElem[8, 16, 11, 9, 12, 7, 13, 9, 11, 16]
 f vector: ZZRingElem[20, 86, 128, 62]
 order of the orbit 60
 order of the stabilizer: 2
 type of the stabilizer: C2
 generators of the stabilizer: PermGroupElem[(1,4)(3,5)]

index: 44
 metric: QQFieldElem[6, 14, 11, 9, 12, 13, 11, 7, 9, 12]
 f vector: ZZRingElem[20, 88, 134, 66]
 order of the orbit 120
 order of the stabilizer: 1
 type of the stabilizer: 1
 generators of the stabilizer: PermGroupElem[]

index: 45
 metric: QQFieldElem[6, 14, 13, 9, 12, 11, 11, 5, 9, 10]
 f vector: ZZRingElem[20, 88, 134, 66]
 order of the orbit 60
 order of the stabilizer: 2
 type of the stabilizer: C2
 generators of the stabilizer: PermGroupElem[(1,3)(2,4)]

index: 46
 metric: QQFieldElem[8, 16, 13, 9, 12, 13, 13, 7, 11, 14]
 f vector: ZZRingElem[20, 88, 134, 66]
 order of the orbit 120
 order of the stabilizer: 1
 type of the stabilizer: 1
 generators of the stabilizer: PermGroupElem[]

index: 47
 metric: QQFieldElem[4, 9, 7, 6, 7, 9, 6, 4, 5, 7]
 f vector: ZZRingElem[20, 88, 134, 66]
 order of the orbit 120
 order of the stabilizer: 1
 type of the stabilizer: 1
 generators of the stabilizer: PermGroupElem[]

```

index: 48
metric: QQFieldElem[6, 16, 13, 11, 14, 15, 9, 7, 9, 12]
f vector: ZZRingElem[20, 88, 134, 66]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 49
metric: QQFieldElem[8, 18, 13, 11, 14, 17, 11, 9, 11, 16]
f vector: ZZRingElem[20, 88, 134, 66]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 50
metric: QQFieldElem[8, 18, 13, 13, 14, 17, 13, 9, 9, 14]
f vector: ZZRingElem[20, 88, 134, 66]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 51
metric: QQFieldElem[8, 16, 17, 9, 16, 13, 13, 7, 11, 14]
f vector: ZZRingElem[20, 88, 134, 66]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 52
metric: QQFieldElem[8, 18, 15, 9, 14, 11, 13, 7, 13, 16]
f vector: ZZRingElem[20, 88, 134, 66]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 53
metric: QQFieldElem[10, 20, 15, 11, 14, 13, 17, 9, 13, 18]
f vector: ZZRingElem[20, 88, 134, 66]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

```

index: 54
metric: QQFieldElem[8, 18, 13, 9, 14, 13, 13, 9, 13, 18]
f vector: ZZRingElem[20, 88, 134, 66]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 55
metric: QQFieldElem[5, 9, 6, 5, 7, 7, 6, 5, 6, 9]
f vector: ZZRingElem[20, 88, 134, 66]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 56
metric: QQFieldElem[4, 8, 6, 4, 7, 5, 5, 4, 6, 8]
f vector: ZZRingElem[20, 88, 134, 66]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 57
metric: QQFieldElem[6, 11, 7, 6, 8, 7, 8, 6, 7, 11]
f vector: ZZRingElem[20, 88, 134, 66]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 58
metric: QQFieldElem[12, 22, 15, 11, 16, 15, 15, 11, 15, 22]
f vector: ZZRingElem[20, 88, 134, 66]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 59
metric: QQFieldElem[8, 16, 11, 9, 12, 15, 9, 9, 11, 16]
f vector: ZZRingElem[20, 88, 134, 66]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

```

index: 60
metric: QQFieldElem[6, 10, 6, 6, 7, 8, 6, 6, 6, 10]
f vector: ZZRingElem[20, 88, 134, 66]
order of the orbit 60
order of the stabilizer: 2
type of the stabilizer: C2
generators of the stabilizer: PermGroupElem[(1,5)(3,4)]

index: 61
metric: QQFieldElem[8, 18, 13, 11, 14, 17, 7, 9, 11, 16]
f vector: ZZRingElem[20, 88, 134, 66]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 62
metric: QQFieldElem[10, 20, 13, 13, 14, 19, 11, 11, 11, 18]
f vector: ZZRingElem[20, 88, 134, 66]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

index: 63
metric: QQFieldElem[12, 22, 23, 11, 22, 15, 19, 11, 15, 22]
f vector: ZZRingElem[20, 88, 134, 66]
order of the orbit 60
order of the stabilizer: 2
type of the stabilizer: C2
generators of the stabilizer: PermGroupElem[(2,5)(3,4)]

index: 64
metric: QQFieldElem[12, 26, 19, 15, 18, 11, 23, 11, 15, 22]
f vector: ZZRingElem[20, 88, 134, 66]
order of the orbit 60
order of the stabilizer: 2
type of the stabilizer: C2
generators of the stabilizer: PermGroupElem[(1,3)(2,4)]

index: 65
metric: QQFieldElem[10, 20, 13, 11, 14, 11, 17, 11, 13, 20]
f vector: ZZRingElem[20, 88, 134, 66]
order of the orbit 120
order of the stabilizer: 1
type of the stabilizer: 1
generators of the stabilizer: PermGroupElem[]

```

[27]: 5661

```
[28]: sorted_fvecs = sort(collect(keys(sorted_collection)))
      f_counter = 1
      for fvec in sorted_fvecs
        println("f vector: ", Vector{Int}(fvec), " has ",
          ↪length(sorted_collection[fvec]), " polytopes")
        #println(length(sorted_collection[fvec]))
        inner_counter = 1
        for i in 1:length(sorted_collection[fvec])
          current_metric, group = collect(sorted_collection[fvec])[i]
          int_metric = [Int(m) for m in current_metric]
          println("index: ", f_counter, ".", inner_counter, " metric: ",
            ↪int_metric, " group: ", group )
          inner_counter += 1
        end
        println()
        f_counter += 1
      end
```

f vector: [20, 60, 70, 30] has 1 polytopes
index: 1.1 metric: [2, 2, 2, 2, 2, 2, 2, 2, 2, 2] group: S5

f vector: [20, 72, 94, 42] has 2 polytopes
index: 2.1 metric: [2, 3, 3, 2, 3, 3, 2, 2, 3, 3] group: D12
index: 2.2 metric: [3, 4, 3, 3, 3, 4, 4, 3, 3, 4] group: D12

f vector: [20, 76, 102, 46] has 1 polytopes
index: 3.1 metric: [4, 6, 5, 5, 6, 5, 5, 5, 5, 6] group: C2 x C2

f vector: [20, 80, 112, 52] has 3 polytopes
index: 4.1 metric: [4, 8, 7, 5, 8, 7, 5, 5, 7, 8] group: C2
index: 4.2 metric: [3, 5, 4, 3, 4, 5, 4, 3, 4, 5] group: C2
index: 4.3 metric: [6, 10, 7, 7, 8, 9, 9, 7, 7, 10] group: C2

f vector: [20, 80, 114, 54] has 4 polytopes
index: 5.1 metric: [2, 4, 4, 3, 4, 4, 3, 2, 3, 3] group: D8
index: 5.2 metric: [4, 8, 7, 7, 8, 7, 7, 5, 5, 6] group: C2 x C2
index: 5.3 metric: [3, 4, 3, 3, 4, 3, 3, 3, 3, 4] group: C2 x C2
index: 5.4 metric: [10, 14, 9, 9, 10, 11, 11, 9, 9, 14] group: D8

f vector: [20, 82, 116, 54] has 1 polytopes
index: 6.1 metric: [3, 5, 4, 4, 4, 5, 3, 3, 3, 4] group: C2

f vector: [20, 84, 122, 58] has 7 polytopes
index: 7.1 metric: [6, 10, 9, 7, 8, 7, 9, 5, 7, 8] group: C2
index: 7.2 metric: [3, 6, 5, 4, 5, 6, 3, 3, 4, 5] group: 1
index: 7.3 metric: [4, 7, 5, 5, 5, 7, 5, 4, 4, 6] group: 1

index: 7.4 metric: [6, 12, 9, 9, 10, 11, 7, 7, 7, 10] group: 1
index: 7.5 metric: [6, 12, 9, 7, 10, 7, 9, 7, 9, 12] group: 1
index: 7.6 metric: [6, 10, 7, 7, 8, 9, 5, 7, 7, 10] group: C2
index: 7.7 metric: [6, 12, 9, 7, 10, 11, 9, 7, 9, 12] group: 1

f vector: [20, 84, 124, 60] has 4 polytopes

index: 8.1 metric: [4, 10, 9, 7, 10, 9, 7, 5, 7, 8] group: C2
index: 8.2 metric: [6, 12, 9, 9, 10, 11, 11, 7, 7, 10] group: C2
index: 8.3 metric: [3, 5, 4, 3, 5, 4, 3, 3, 4, 5] group: C2
index: 8.4 metric: [5, 8, 5, 5, 6, 6, 6, 5, 5, 8] group: C2

f vector: [20, 86, 128, 62] has 20 polytopes

index: 9.1 metric: [3, 6, 5, 4, 5, 6, 5, 3, 4, 5] group: C2
index: 9.2 metric: [6, 12, 11, 7, 10, 9, 9, 5, 9, 10] group: 1
index: 9.3 metric: [8, 14, 11, 9, 10, 11, 13, 7, 9, 12] group: C2
index: 9.4 metric: [3, 7, 6, 5, 6, 7, 4, 3, 4, 5] group: C2
index: 9.5 metric: [4, 8, 6, 5, 6, 8, 5, 4, 5, 7] group: 1
index: 9.6 metric: [2, 4, 3, 3, 3, 4, 3, 2, 2, 3] group: 1
index: 9.7 metric: [6, 14, 11, 9, 12, 13, 7, 7, 9, 12] group: 1
index: 9.8 metric: [6, 14, 11, 11, 12, 13, 9, 7, 7, 10] group: 1
index: 9.9 metric: [8, 16, 11, 11, 12, 15, 11, 9, 9, 14] group: 1
index: 9.10 metric: [8, 14, 15, 9, 14, 11, 13, 7, 9, 12] group: C2
index: 9.11 metric: [8, 16, 13, 9, 12, 9, 13, 7, 11, 14] group: 1
index: 9.12 metric: [6, 14, 11, 7, 12, 9, 9, 7, 11, 14] group: 1
index: 9.13 metric: [8, 16, 11, 9, 12, 11, 13, 9, 11, 16] group: 1
index: 9.14 metric: [10, 16, 11, 9, 12, 13, 11, 9, 11, 16] group: C2
index: 9.15 metric: [4, 7, 5, 4, 6, 4, 5, 4, 5, 7] group: 1
index: 9.16 metric: [12, 20, 13, 11, 14, 13, 15, 11, 13, 20] group: C2
index: 9.17 metric: [6, 12, 9, 7, 10, 11, 5, 7, 9, 12] group: C2
index: 9.18 metric: [8, 14, 9, 9, 10, 13, 9, 9, 9, 14] group: 1
index: 9.19 metric: [8, 16, 11, 11, 12, 15, 7, 9, 9, 14] group: 1
index: 9.20 metric: [8, 16, 11, 9, 12, 7, 13, 9, 11, 16] group: C2

f vector: [20, 88, 134, 66] has 22 polytopes

index: 10.1 metric: [6, 14, 11, 9, 12, 13, 11, 7, 9, 12] group: 1
index: 10.2 metric: [6, 14, 13, 9, 12, 11, 11, 5, 9, 10] group: C2
index: 10.3 metric: [8, 16, 13, 9, 12, 13, 13, 7, 11, 14] group: 1
index: 10.4 metric: [4, 9, 7, 6, 7, 9, 6, 4, 5, 7] group: 1
index: 10.5 metric: [6, 16, 13, 11, 14, 15, 9, 7, 9, 12] group: 1
index: 10.6 metric: [8, 18, 13, 11, 14, 17, 11, 9, 11, 16] group: 1
index: 10.7 metric: [8, 18, 13, 13, 14, 17, 13, 9, 9, 14] group: 1
index: 10.8 metric: [8, 16, 17, 9, 16, 13, 13, 7, 11, 14] group: 1
index: 10.9 metric: [8, 18, 15, 9, 14, 11, 13, 7, 13, 16] group: 1
index: 10.10 metric: [10, 20, 15, 11, 14, 13, 17, 9, 13, 18] group: 1
index: 10.11 metric: [8, 18, 13, 9, 14, 13, 13, 9, 13, 18] group: 1
index: 10.12 metric: [5, 9, 6, 5, 7, 7, 6, 5, 6, 9] group: 1
index: 10.13 metric: [4, 8, 6, 4, 7, 5, 5, 4, 6, 8] group: 1
index: 10.14 metric: [6, 11, 7, 6, 8, 7, 8, 6, 7, 11] group: 1

```

index: 10.15 metric: [12, 22, 15, 11, 16, 15, 15, 11, 15, 22] group: 1
index: 10.16 metric: [8, 16, 11, 9, 12, 15, 9, 9, 11, 16] group: 1
index: 10.17 metric: [6, 10, 6, 6, 7, 8, 6, 6, 6, 10] group: C2
index: 10.18 metric: [8, 18, 13, 11, 14, 17, 7, 9, 11, 16] group: 1
index: 10.19 metric: [10, 20, 13, 13, 14, 19, 11, 11, 11, 18] group: 1
index: 10.20 metric: [12, 22, 23, 11, 22, 15, 19, 11, 15, 22] group: C2
index: 10.21 metric: [12, 26, 19, 15, 18, 11, 23, 11, 15, 22] group: C2
index: 10.22 metric: [10, 20, 13, 11, 14, 11, 17, 11, 13, 20] group: 1

```

We save the polytopes with their corresponding metrics in a JSON file in the Zenodo data base [2].

```
[29]: save("Polytopes_strict_metrics_5.mrdi",S5)
```

```
[30]: load("Polytopes_strict_metrics_5.mrdi")
```

```

[30]: 65-element Vector{Tuple{Polyhedron{QQFieldElem}, Vector{QQFieldElem}}}:
 (Polytope in ambient dimension 5, [2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
 (Polytope in ambient dimension 5, [2, 3, 3, 2, 3, 3, 2, 2, 3, 3])
 (Polytope in ambient dimension 5, [4, 6, 5, 5, 6, 5, 5, 5, 5, 6])
 (Polytope in ambient dimension 5, [3, 4, 3, 3, 3, 4, 4, 3, 3, 4])
 (Polytope in ambient dimension 5, [3, 5, 4, 4, 4, 5, 3, 3, 3, 4])
 (Polytope in ambient dimension 5, [2, 4, 4, 3, 4, 4, 3, 2, 3, 3])
 (Polytope in ambient dimension 5, [4, 8, 7, 5, 8, 7, 5, 5, 7, 8])
 (Polytope in ambient dimension 5, [4, 8, 7, 7, 8, 7, 7, 5, 5, 6])
 (Polytope in ambient dimension 5, [3, 5, 4, 3, 4, 5, 4, 3, 4, 5])
 (Polytope in ambient dimension 5, [6, 10, 7, 7, 8, 9, 9, 7, 7, 10])
 (Polytope in ambient dimension 5, [6, 10, 9, 7, 8, 7, 9, 5, 7, 8])
 (Polytope in ambient dimension 5, [3, 6, 5, 4, 5, 6, 3, 3, 4, 5])
 (Polytope in ambient dimension 5, [4, 7, 5, 5, 5, 7, 5, 4, 4, 6])
 (Polytope in ambient dimension 5, [6, 12, 9, 9, 10, 11, 7, 7, 7, 10])
 (Polytope in ambient dimension 5, [6, 12, 9, 7, 10, 7, 9, 7, 9, 12])
 (Polytope in ambient dimension 5, [3, 4, 3, 3, 4, 3, 3, 3, 3, 4])
 (Polytope in ambient dimension 5, [10, 14, 9, 9, 10, 11, 11, 9, 9, 14])
 (Polytope in ambient dimension 5, [6, 10, 7, 7, 8, 9, 5, 7, 7, 10])
 (Polytope in ambient dimension 5, [4, 10, 9, 7, 10, 9, 7, 5, 7, 8])
 (Polytope in ambient dimension 5, [3, 6, 5, 4, 5, 6, 5, 3, 4, 5])
 (Polytope in ambient dimension 5, [6, 12, 9, 7, 10, 11, 9, 7, 9, 12])
 (Polytope in ambient dimension 5, [6, 12, 9, 9, 10, 11, 11, 7, 7, 10])
 (Polytope in ambient dimension 5, [6, 12, 11, 7, 10, 9, 9, 5, 9, 10])

 (Polytope in ambient dimension 5, [8, 16, 11, 9, 12, 7, 13, 9, 11, 16])
 (Polytope in ambient dimension 5, [6, 14, 11, 9, 12, 13, 11, 7, 9, 12])
 (Polytope in ambient dimension 5, [6, 14, 13, 9, 12, 11, 11, 5, 9, 10])
 (Polytope in ambient dimension 5, [8, 16, 13, 9, 12, 13, 13, 7, 11, 14])
 (Polytope in ambient dimension 5, [4, 9, 7, 6, 7, 9, 6, 4, 5, 7])
 (Polytope in ambient dimension 5, [6, 16, 13, 11, 14, 15, 9, 7, 9, 12])
 (Polytope in ambient dimension 5, [8, 18, 13, 11, 14, 17, 11, 9, 11, 16])

```

```

(Polytope in ambient dimension 5, [8, 18, 13, 13, 14, 17, 13, 9, 9, 14])
(Polytope in ambient dimension 5, [8, 16, 17, 9, 16, 13, 13, 7, 11, 14])
(Polytope in ambient dimension 5, [8, 18, 15, 9, 14, 11, 13, 7, 13, 16])
(Polytope in ambient dimension 5, [10, 20, 15, 11, 14, 13, 17, 9, 13, 18])
(Polytope in ambient dimension 5, [8, 18, 13, 9, 14, 13, 13, 9, 13, 18])
(Polytope in ambient dimension 5, [5, 9, 6, 5, 7, 7, 6, 5, 6, 9])
(Polytope in ambient dimension 5, [4, 8, 6, 4, 7, 5, 5, 4, 6, 8])
(Polytope in ambient dimension 5, [6, 11, 7, 6, 8, 7, 8, 6, 7, 11])
(Polytope in ambient dimension 5, [12, 22, 15, 11, 16, 15, 15, 11, 15, 22])
(Polytope in ambient dimension 5, [8, 16, 11, 9, 12, 15, 9, 9, 11, 16])
(Polytope in ambient dimension 5, [6, 10, 6, 6, 7, 8, 6, 6, 6, 10])
(Polytope in ambient dimension 5, [8, 18, 13, 11, 14, 17, 7, 9, 11, 16])
(Polytope in ambient dimension 5, [10, 20, 13, 13, 14, 19, 11, 11, 11, 18])
(Polytope in ambient dimension 5, [12, 22, 23, 11, 22, 15, 19, 11, 15, 22])
(Polytope in ambient dimension 5, [12, 26, 19, 15, 18, 11, 23, 11, 15, 22])
(Polytope in ambient dimension 5, [10, 20, 13, 11, 14, 11, 17, 11, 13, 20])

```

To visualize the polytopes, one can compute the Schlegel diagrams. As noted in <https://mathrepo.mis.mpg.de/OSCAR/OscarPolytopes.html>, the command to visualize a Polymake polytope is `Polymake.polytope.visual(P)`, but this command has issues in a Jupyter notebook like this one. It can be used in the Julia command line.

References

- [1] Delucchi, E., Kühne, L., Mühlherr, L. (2025), *Combinatorial invariants of finite metric spaces and the Wasserstein distance*, 2025, <https://arxiv.org/abs/2408.15584>.
- [2] Delucchi, E., Kühne, L., Mühlherr, L., & Rambau, J. (2024). *Dataset of Kantorovich-Rubinstein-Wasserstein Polytopes of Metric Spaces on up to 6 Points (topcom 1.1.5, Julia)*. Zenodo. <https://doi.org/10.5281/zenodo.12773907>
- [3] Mühlherr L. (2026) *A Graph Theoretical Perspective on Hyperplane Arrangement Theory*. Bielefeld: Universität Bielefeld. <https://doi.org/10.4119/unibi/3013025>
- [4] Rambau, J.(2023). *Symmetric lexicographic subset reverse search for the enumeration of circuits, cocircuits, and fundamental circuits*, https://www.wm.uni-bayreuth.de/de/team/rambau_joerg/TOPCOMSymLexSubsetRS-2.pdf.